

Projet SE 2 — Multithreading dans Nachos

Lilian Limousy — Kousay Amara
Université de Bordeaux — UF Informatique
10 novembre 2025

Table des matières

1. Introduction / Bilan
2. Points délicats
 - 2.1. Création et initialisation des threads
 - 2.2. Gestion des piles utilisateur
 - 2.3. Synchronisation de la console
3. Limitations
4. Tests
5. Conclusion

1. Introduction / Bilan

Ce second devoir visait à implémenter le multithreading utilisateur dans Nachos. Nous avons ajouté deux appels système : **ThreadCreate()** et **ThreadExit()**. Les threads partagent le même espace d'adressage et disposent chacun d'une pile indépendante allouée dynamiquement.

Travail accompli :

- Allocation des piles via un bitmap.
- Gestion du compteur de threads pour la terminaison automatique.
- Tests validant la création simultanée de plusieurs threads.

Travail non finalisé :

La synchronisation de la console : les sorties s'entremêlent malgré l'utilisation de locks et de sémaphores. Hormis cela, le système de multithreading fonctionne correctement.

2. Points délicats

2.1 Création et initialisation des threads :

Chaque thread utilisateur est propulsé par un thread noyau distinct. **do_ThreadCreate()** alloue une pile libre, prépare les registres et lance **StartUserThread()**, qui initialise les registres MIPS avant **machine->Run()**.

2.2 Gestion des piles utilisateur :

La zone **UserStacksAreaSize = 2048 octets** est divisée en 8 piles de 256 octets gérées par bitmap.

AllocateUserStack() attribue un slot et **FreeUserStack()** le libère lors de **ThreadExit()**.

2.3 Synchronisation de la console :

Un sémaphore **consoleMutex** protège les accès à la console, mais comme **PutString()** appelle plusieurs **PutChar()**, des interruptions peuvent encore provoquer un affichage entremêlé.

3. Limitations

- Synchronisation console : L'affichage des messages mélangés.
- Taille de pile fixe (256 octets)
- Nombre maximal de threads limité à 8.

Les limitations de synchronisation restent les plus problématiques.

4. Tests

Tests réalisés :

- **many_threads.c** : 6 threads affichant chacun des boucles.
- **multiple_threads_test.c** : création et terminaison enchaînée.

Exécution typique :

```
./nachos -rs 1234 -x ../test/many_threads
=== Testing many threads with bitmap allocation ===
Created threads 1...6
Thread 2: loop 0
Thread 5: loop 1
...
Machine going down!
```

Les threads se créent et s'exécutent bien ; la console reste le seul point non fiable.

5. Conclusion

L'implémentation est fonctionnelle et stable, démontrant la cohabitation de plusieurs threads utilisateur. La gestion des piles et la terminaison automatique sont maîtrisées. Le principal point faible reste la synchronisation des affichages.